

# The Shed

Pilot deployment plan for the CMR Specialist (v2.0 - office use)

Take the proven system exactly as it is - no new features - and give it a safe, locked, backed-up home where office colleagues can use it daily. Same console, same workflows. Just no longer on a laptop.

Analogy: a good generator sits in your bedroom on your power, with one shared key to the front door. The shed moves it outside into a locked shed with its own keys, nightly photocopies of the papers, and instructions pinned on the door.

## 1. The five pieces

---

### Piece 1 - A home: one always-on machine

A spare office PC, mini-PC, or small rented server (2 CPUs / 4 GB RAM is plenty). It runs the same two programs (web + worker) and starts them automatically on boot. From that day, nobody depends on any individual's laptop.

### Piece 2 - The locks: a login system

Every staff member gets a username and password and signs in through the console. Three ideas inside:

- **Identity** - 'who are you?' Every case, approval, and upload carries a real name. Anonymous approvals are theater; named approvals are accountability.
- **Session** - 'the hand stamp.' One login gives the browser a temporary token; no password-typing per click. It expires (e.g. end of day) so forgotten browsers stop being trusted.
- **Revocation** - 'changing the lock.' Disable the account when someone leaves; their access dies instantly. No shared secret to rotate, no memo to the whole office.

Locks are not encryption (HTTPS, also included via a free Caddy front door) and not yet permissions - every logged-in staff member can do everything. Roles come later.

### Piece 3 - A safe: Postgres plus nightly backup

Flip one setting (DATABASE\_URL) from the current SQLite file to Postgres - the compose file already exists in the repo. One scheduled nightly job copies the database to a second location: worst case after a failure is losing a day, never everything. A 90-day auto-purge rides on the same mechanism so old personal data deletes itself.

### Piece 4 - The rules: one page on data

Written, not coded: what the system stores (names, plates, NINs, receipts), who may see it (pilot team only), how long it lives (90 days), who approves exceptions. Staff acknowledge it. Cheapest piece, biggest legal protection.

### Piece 5 - The desk sheet: one page for staff

How to open the console. Three statuses in plain words: *needs your input* (answer the prompt), *needs approval* (a human must confirm), *fallback badge* (the AI was quiet - check the result twice). One phone number for problems.

## 2. A day in the shed

---

- Morning: everything is already running. Staff open a browser tab and work cases.
- Payment reconciliation first (read-only, safest), ownership transfers second. Nothing WRITE-gated until the approval habit exists.
- The screen reports its own problems (badges, prompts, queue states); nobody reads logs.
- Night: the backup runs alone. Nobody maintains anything.

Cost and time: software effectively free (compose file, Caddy, Postgres all in-repo or free). About a day of setup plus half a day of staff walkthrough. Daily AI cost: cents.

### 3. Git workflow (standing rule)

All shed work is built on **branches**, never by rebuilding. *main* always boots; each job gets a branch (shed/login-system, shed/postgres-backup, ...), merges after tests pass, then the branch is deleted. When the shed is done and running, *main* is tagged as the new version. Commits stay small and readable; *.env* is never committed.

```
git checkout -b shed/login-system # workspace per job

# ... work, commit small, push, open PR against main ...

# tests green -> merge -> delete branch

git tag v2.1-shed # version = milestone, when the shed ships
```

### 4. Noted future UI work (not in the shed)

| UI                    | Purpose                                                      | Guardrail                                                         |
|-----------------------|--------------------------------------------------------------|-------------------------------------------------------------------|
| MockDB viewer/editor  | Browse + insert test data (RRRs, plates, NINs) without SQL   | TEST-labeled, synthetic rows only; never wired to a real registry |
| Harness DB viewer     | Read-only trace of cases, fields, tasks, steps, audits       | Read-only: all state changes stay in the API                      |
| Worker monitor        | Liveness, leased task, queue depth, recent outcomes          | Read-only; no log files needed                                    |
| Bulk knowledge upload | Many PDFs/docs at once, live counter (uploaded vs published) | Upload stages; publishing still required per version              |

Build order when the time comes: mockDB editor first (unblocks test setup), harness viewer second, worker monitor third, bulk upload alongside knowledge work.

### 5. What stays out - and why

No citizen portal, supervisor dashboard, live-update plumbing, or second API version. Each is a guess about what staff will need. The shed converts guesses into requests: after a month of real use, the team will say exactly which piece of the target architecture to build first - and they will be right, because they will be speaking from the work.